isfound = false;

nums = { 2, 4, 3, 6, 8, 11, 15 } target = 9

0 1 2 3 4 5 6

p_1 p_2

p_1 / → for (p_1 → p_2) push → ans.

isfound = true;

```
p1 = 0;
int sum = 0;
while (p1 < n-1)
{
    if (isfound) { break; }
    p2 = p1+1, sum = 0;
    while (p2 < n)
    {
        if (sum == target)
        {
            for( p1 -> p2) { push_back() }   X ⇒ ans.push_back(p1)
                                                 ans.push_back(p2)
            isfound = true;
            break;
        }
        sum += nums[i];
        p2++;
    }
    p1++;
}
```

0 1 2

[3, 2, 4] target = 6

⇒ p_1 p_2 p_2

p_1 p_2

sum = 3+2 = 5

3+4 = 7

2+4 = 6

0 1 2

[3, 2, 3] target = 6

[0, 2] ans

for (p1 → p2) X

{

TC → $O(n^2)$

optimization

⇒ Using Hashmap ⇒

0 1 2 3

[2, 7, 11, 15] target = 9

9-2 = 7 9-7 = 2

Hashmap element : index

2 : 0

→ check 7 is in map or not ?

if not insert → ele = 2 & index = 0

2 is present ⇒ [0, 1] ans

T.C = O(n)

0 1 2 3 4 5 6

[2, 4, 3, 6, 8, 11, 15] target = 9

9-2 = 7 9-4 = 5 9-3 = 6 9-6 = 3 → index = 3

[2, 3]

Map

ele : index

2 : 0

4 : 1

3 : 2

```
unordered_map<int, int> mp;
    vector<int> ans(2);
for(i ------> n)
{ int diff = target - nums[i];
    if ( mp.find(diff) != mp.end() )
    {   int val = mp.at(diff);
        ans[0] = val;
        ans[1] = i;
        break;
    }
    // insert into map.
    mp.insert ({nums[i], i});
}
return ans;
```

→ 2 size but through this initialization [0, 0] 0 present at index 0 & 1.

key is present

return value corresponding to key.